

UEFI SUBSYSTEMS & THE INTEL HDA DRIVER

PROTOCOL INVENTORY AND DRIVER ARCHITECTURE FOR RUEFI

AUTHOR

Javier Fernández

CONTACT

jfernand@me.com

ISSUED

2026-09-15

ABSTRACT

ruefi is a bare-metal Rust UEFI application: a ratatui platform explorer (memory map, ACPI tables, PCI bus, disks, NVRAM variables) plus a GOP-driven arcade game, and — as of this report — a from-scratch Intel High Definition Audio driver split into a hardware-agnostic core (`intel-hda`) and a UEFI backend (`intel-hda-uefi`). None of this runs on top of an operating system; every capability above comes from a UEFI protocol or boot/runtime service called directly. Part I inventories every one of those interfaces actually present in the source tree — what it is, which crate module names it, and which file in this repo calls it. Part II documents the HDA driver's own architecture: why it's split the way it is, the hardware model it's built against, its bring-up sequence, and two real emulation-specific bugs found and fixed while getting a test tone to play correctly under QEMU.

DOCUMENT CONTROL

REPOSITORY	ruefi — bare-metal Rust, <code>#![no_std]</code> , <code>x86_64-unknown-uefi</code>
UEFI CRATE	<code>uefi 0.40 (rust-osdev/uefi-rs)</code>
TEST TARGET	<code>QEMU 8.2 + OVMF</code> , <code>q35 machine</code> , <code>ich9-intel-hda</code> + <code>hda-duplex</code>
STATUS	Audio tab plays a verified 440 Hz test tone; see Section 11

01

SCOPE AND READING ORDER

PART I	Every UEFI protocol and boot/runtime service this codebase calls: what it is, where, and why.
§ 2	The boot/runtime split and the protocol database — the two ideas everything else in Part I sits on top of.
§ 3	Console I/O: how a ratatui Backend and a game loop both get keys and pixels out of firmware alone.
§ 4	Bus and device access: PCI config space, block I/O, ACPI tables, NVRAM variables, the memory map.
§ 5	Timers, events, and DMA memory — the primitives the HDA driver builds on.
APPENDIX A	Every other UEFI/PI protocol that exists but this codebase doesn't use — the full spec surface, for context.
PART II	The Intel HDA driver: crate layout, hardware model, bring-up, and what broke against QEMU.

COMPANION DOCUMENT

This report covers **interfaces**, not register-level detail. For the full Intel HDA register map, verb encoding, and codec parameter tables, see docs/hda-register-map.typ in this same directory — Part II here summarizes and cross-references it rather than repeating it.

02

THE BOOT/RUNTIME SPLIT AND THE PROTOCOL DATABASE

UEFI firmware exposes almost everything through a single mechanism: a runtime **protocol database** mapping opaque `Handle`s to typed interfaces identified by GUID. “Does this PCI function support the Block I/O protocol?” and “give me the Graphics Output protocol on whatever handle has one” are the two questions every driver in this codebase asks, over and over, via `uefi::boot::locate_handle_buffer` and `uefi::boot::open_protocol`.

SERVICE CLASS	CHARACTERISTICS
Boot Services	Everything under <code>uefi::boot</code> — protocol location/open, memory allocation, events/timers, stalling. Available only before <code>ExitBootServices</code> is called; ruefi never calls it, so the whole app lives entirely in this phase.
Runtime Services	Everything under <code>uefi::runtime</code> — NVRAM variable access here, also time/reset/capsule services elsewhere. Guaranteed to keep working even after an OS has taken over boot services, which is why variable access goes through this separate namespace.
Configuration tables	A firmware-provided array of (GUID, address) pairs — <code>uefi::system::with_config_table</code> — used to hand a caller a pointer to a data structure (ACPI's RSDP) rather than a full protocol interface.

Every protocol open call in this codebase uses `OpenProtocolAttributes::GetProtocol` rather than `open_protocol_exclusive`, with one deliberate exception (the GOP, Section 3.2). `GetProtocol` is non-exclusive: it doesn't force the firmware to disconnect whatever driver is already bound to that handle. For PCI and Block I/O, that matters concretely — exclusive access to a root bridge or the boot disk's controller would force-disconnect the very driver serving the media the app booted from.

03

CONSOLE I/O

3.1

SIMPLE TEXT OUTPUT / SIMPLE TEXT INPUT

INTERFACE `uefi::proto::console::text::Output` / `Input`, reached via `uefi::system::with_stdout` / `with_stdin` rather than an explicit `open_protocol` call

USED IN `src/uefi_backend.rs` (all of it); `src/app.rs` (`App::new`, `next_tick`, `on_key`); `src/bin/asteroids/main.rs` (key handling)

PURPOSE The entire `ratatui` rendering surface, and all keyboard input for both the TUI app and the asteroids game

`UefiBackend` in `src/uefi_backend.rs` implements `ratatui`'s `Backend` trait directly against the firmware text console: it queries available text modes and picks one at construction, then on every `draw` call walks the diffed cell buffer and, per run of same-colored cells, calls `set_cursor_position` followed by `output_string_lossy`. Cursor visibility, position queries, and screen clearing all go through the same `with_stdout` closure pattern.

SRC/UEFI_BACKEND.RS — ONE CELL RUN, ABBREVIATED

```
system::with_stdout(|stdout| {
    stdout.set_cursor_position(run_x as usize, run_y as usize)
});
system::with_stdout(|stdout| stdout.output_string_lossy(s));
```

Key input is event-driven rather than polled: `App::new` calls `stdin.wait_for_key_event()` once to obtain a firmware `Event` signaled whenever a keystroke is available, then `next_tick` (Section 5.1) waits on that event alongside a periodic timer and, once it fires, calls `stdin.read_key()` to actually consume the keystroke. The asteroids game loop (`src/bin/asteroids/main.rs`) uses the identical pattern for movement/fire/quit input.

3.2 GRAPHICS OUTPUT PROTOCOL (GOP)

INTERFACE `uefi::proto::console::gop::GraphicsOutput`, `PixelFormat`

USED IN `src/bin/asteroids/main.rs` (`open_gop`), `src/bin/asteroids/gop_display.rs` (`GopDisplay`)

PURPOSE Sole display backend for the asteroids game — direct framebuffer pixel access, not the text console

This is the one place in the codebase that opens a protocol **exclusively** (`boot::open_protocol_exclusive::<GraphicsOutput>`, reached via `boot::get_handle_for_protocol`): the game owns the whole display for its lifetime, so there's no other consumer to avoid disconnecting.

`GopDisplay::new` reads the current mode's resolution, stride, and pixel format once, then `present()` writes directly into `gop.frame_buffer()` with `core::ptr::copy_nonoverlapping` row by row.

WHY NOT GOP'S BLT

The file's own header comment explains the choice explicitly: GOP exposes a `Blt` (block transfer) operation that can copy a buffer to video, but OVMF's software implementation of it is measurably slower than writing into the mapped framebuffer directly with `ptr::copy_nonoverlapping`. For a real-time game loop redrawing every frame, that difference is the whole reason to bypass `Blt`.

04 BUS AND DEVICE ACCESS

4.1 PCI ROOT BRIDGE I/O PROTOCOL

INTERFACE `uefi::proto::pci::root_bridge::PciRootBridgeIo`, `uefi::proto::pci::PciIoAddress`

USED IN `src/explore/pci.rs` (`scan`); `crates/intel-hda-uefi/src/lib.rs` (`find_and_enable_controller`)

PURPOSE Enumerate PCI config space for the PCI explorer tab, and — separately — find, identify, and enable the Intel HDA controller

Both call sites walk bus 0, devices 0–31, functions 0–7, reading the vendor/device ID register (offset 0x00) to detect a present function and the class-code register (offset 0x08) to classify it, using `root_bridge.pci().read_one::<u32>(addr.with_register(offset))`. The HDA backend additionally **writes** through the same interface — `pci.write_one::<u16>` at offset 0x04, the PCI Command register — to set the

Memory Space and Bus Master bits once it identifies a class 0x04 / subclass 0x03 function, and reads BAR0 (and BAR1, for a 64-bit BAR) to compute the controller's MMIO base address.

CRATES/INTEL-HDA-UEFI/SRC/LIB.RS — ENABLING THE CONTROLLER ONCE FOUND

```
let command = pci.read_one::<u16>(addr.with_register(0x04)).unwrap_or(0);
let _ = pci.write_one::<u16>(
    addr.with_register(0x04),
    command | PCI_COMMAND_MEMORY_SPACE | PCI_COMMAND_BUS_MASTER,
);
```

Past this point, the HDA driver never touches `PciRootBridgeIo` again — the controller's actual registers are reached by dereferencing BAR0's physical address directly as MMIO (Section 8 in Part II), not through the PCI protocol.

4.2 BLOCK I/O PROTOCOL

INTERFACE `uefi::proto::media::block::BlockIO`
 USED IN `src/explore/disks.rs (open, scan, read_first_block)`
 PURPOSE Enumerate disks/partitions for the Disks tab and hex-dump each one's first sector

`scan()` locates every handle supporting `BlockIO`, opens each non-exclusively, and reads `block_io.media()` for block size, block count, and whether the device is a logical partition or a raw physical disk. `read_first_block()` calls `block_io.read_blocks(media_id, 0, &mut buf)` to pull LBA 0 for the hex-dump dialog — the MBR/GPT header, when present.

4.3 ACPI CONFIGURATION TABLE

INTERFACE `uefi::table::cfg::ConfigTableEntry`, via `uefi::system::with_config_table`
 USED IN `src/explore/acpi.rs (discover)`
 PURPOSE Locate the RSDP so the ACPI tab can walk and hex-dump the table chain

NOT THE ACPITABLE PROTOCOL

The file's header comment is explicit about this: UEFI also defines an `AcpiTable protocol`, but that one exists for a driver to **install** new ACPI tables into a running system — not to read the ones already there. Reading starts and ends with the configuration-table lookup; everything after (RSDP → XSDT/RSDT → each table's header) is hand-rolled pointer arithmetic over physical memory, exactly as any ACPI-consuming OS component would do it.

`discover()` looks up `ConfigTableEntry::ACPI2_GUID` first, falling back to `ConfigTableEntry::ACPI_GUID` for older firmware, to get the RSDP's physical address, then walks the resulting table chain manually.

4.4 RUNTIME VARIABLE SERVICES

INTERFACE `uefi::runtime::{variable_keys, get_variable_boxed, VariableAttributes, VariableKey, VariableVendor}`
 USED IN `src/explore/vars.rs (list, UefiVariable::read); src/explore/well_known_vars.rs (pure lookup table, no UEFI calls)`
 PURPOSE Enumerate and read every NVRAM variable for the Variables tab, decode standard attribute bits, and annotate well-known names against the UEFI global namespace

`variable_keys()` wraps `GetNextVariableName` to produce every (name, vendor GUID) pair currently stored; `get_variable_boxed` wraps `GetVariable` to fetch one variable's raw bytes and attributes on demand — called once per key during the initial listing and again whenever the detail dialog opens. `well_known_vars.rs` is a static description table keyed by name, checked against `VariableVendor::GLOBAL_VARIABLE` to distinguish spec-defined variables from vendor-specific ones; it makes no UEFI calls of its own.

4.5 MEMORY MAP

INTERFACE `uefi::boot::memory_map`, `uefi::mem::memory_map::MemoryMap` trait
USED IN `src/explore/memmap.rs (snapshot)`
PURPOSE Power the Memory tab's region list and per-type summary

`boot::memory_map(MemoryType::LOADER_DATA)` returns an owned snapshot, iterated via the `MemoryMap` trait's `entries()` to pull each descriptor's type, physical start, and page count. The module's own doc comment flags a subtlety worth restating here: taking the snapshot can itself allocate, so the raw map is read out into owned `MemRegion` values immediately rather than held onto — anything else that allocates afterward would otherwise invalidate it.

05

TIMERS, EVENTS, AND DMA MEMORY

5.1 EVENT-DRIVEN TICKING

INTERFACE `uefi::boot::{create_event, set_timer, wait_for_event, EventType, TimerTrigger, Tpl}`, `uefi::Event`
USED IN `src/app.rs (App::new, run, next_tick)`; `src/bin/asteroids/main.rs` (same pattern)
PURPOSE The standard UEFI recipe for a loop that reacts to either a periodic tick or an external event, without a real OS scheduler

Both the TUI app and the game create one periodic `EventType::TIMER` event (250ms and 16ms respectively) alongside the console's key-available event, then block on both at once with a single `wait_for_event` call each iteration — the index of whichever fired tells the loop whether to advance a spinner/game-tick or read and dispatch a keystroke.

SRC/APP.RS — THE WAIT, ABBREVIATED

```
let mut events = [
    unsafe { self.key_event.unsafe_clone() },
    unsafe { self.timer_event.unsafe_clone() },
];
let index = boot::wait_for_event(&mut events).unwrap();
```

5.2 STALLING

`uefi::boot::stall` appears twice for two very different reasons: a flat 3-second wait in the asteroids entry point when no GOP is found (giving a human time to read the resulting error before the app exits), and — far more load-bearing — as the entire implementation of `Platform::delay_us` in `crates/intel-hda-uefi/src/lib.rs`, which the HDA driver core calls whenever the Intel HDA spec requires waiting a specific number of microseconds for hardware (or emulated hardware) state to settle: controller reset, stream reset, and CORB/RIRB command polling all go through it.

5.3 DMA-CAPABLE MEMORY

INTERFACE `uefi::boot::allocate_pages`, `AllocateType`, `MemoryType`
USED IN `crates/intel-hda-uefi/src/lib.rs (UefiPlatform::alloc_dma)`
PURPOSE Give the HDA controller physically-contiguous, below-4GB command and sample buffers

`alloc_dma` rounds the requested length up to whole pages and calls `boot::allocate_pages(AllocateType::MaxAddress(0xFFFF_FFFF), MemoryType::LOADER_DATA, pages)`, then zero-fills the result. Two assumptions this leans on, both standard for UEFI boot-time code: the firmware's page tables identity-map all usable RAM, so a pointer to the allocation **is** its physical address with no translation needed (confirmed empirically in Section 11); and staying below the 4GB boundary sidesteps ever needing to check the controller's 64-bit-addressing capability bit at all.

THIS IS A SHORTCUT, NOT A SPEC-CORRECT DMA PATH

A pointer being usable as a bus address is only true because nothing on this test target translates PCI bus-master accesses – QEMU/OVMF here runs with no virtual IOMMU. The UEFI-correct way to get a DMA-safe address is `EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL`’s `Map()` / `Unmap()` / `AllocateBuffer()` / `FreeBuffer()` (mirrored per-device by `EFI_PCI_IO_PROTOCOL`), which return whatever address a device should actually use – identical to the physical address with no IOMMU active, a translated one with an IOMMU (e.g. Intel VT-d) enabled, or a bounce buffer if the device can’t address the memory at all. `uefi-rs 0.40` doesn’t expose these yet (`root_bridge.rs` carries a literal `// TODO: map & unmap & copy memory`), so today’s `alloc_dma` would need to reach past the safe wrapper into `uefi-raw`’s function pointers to do this correctly. On real hardware with pre-boot DMA protection enabled, the current shortcut would point the controller’s DMA engine at the wrong physical memory.

06 APPENDIX A – COMPLETE UEFI/PI PROTOCOL CATALOG

SCOPE AND SOURCING

Part I above covers the handful of protocols this repository actually calls, in depth. This appendix is the opposite: every protocol GUID defined in the UEFI and Platform Initialization (PI) specifications, as implemented by TianoCore EDK2 – the reference implementation OVMF (and most real vendor firmware) is built from. Descriptions are extracted directly from each header’s own file comment in `tianocore/edk2` (`MdePkg`, `MdeModulePkg`, `SecurityPkg`, `NetworkPkg`), not written from memory, so wording here matches upstream rather than paraphrasing it. Categories marked **(firmware-internal, PI spec)** are how DXE/SMM/PEI modules talk to each other **inside** firmware, before or alongside boot services – a boot-time application like `ruefi` cannot reach these; they’re included because they’re still part of “does UEFI have a protocol for X,” just answered at a different layer than an application lives at.

6.1 CONSOLE, INPUT & SERIAL I/O

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
<code>AbsolutePointer</code>	The file provides services that allow information about an absolute pointer device to be retrieved.
<code>SerialIo</code>	Serial IO protocol as defined in the UEFI 2.0 specification. Abstraction of a basic serial device.
<code>SimplePointer</code>	Simple Pointer protocol from the UEFI 2.0 specification. Abstraction of a very simple pointer device like a mouse or trackball.
<code>SimpleTextIn</code>	Simple Text Input protocol from the UEFI 2.0 specification. Abstraction of a very simple input device like a keyboard or serial terminal.
<code>SimpleTextInEx</code>	Simple Text Input Ex protocol from the UEFI 2.0 specification. This protocol defines an extension to the <code>EFI_SIMPLE_TEXT_INPUT_PROTOCOL</code> which exposes much more state and modifier information from the input device, also allows one to register a notification for a particular keystroke.
<code>SimpleTextOut</code>	Simple Text Out protocol from the UEFI 2.0 specification. Abstraction of a very simple text based output device like VGA text mode or a serial terminal.

6.2 GRAPHICS & DISPLAY

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
<code>BootLogo</code>	Boot Logo protocol is used to convey information of Logo displayed during boot.
<code>BootLogo2</code>	Boot Logo 2 Protocol is used to convey information of Logo displayed during boot. The Boot Logo 2 Protocol is a replacement for the Boot Logo Protocol.
<code>DisplayProtocol</code>	FormDisplay protocol to show Form.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
EdidActive	EDID Active Protocol from the UEFI 2.0 specification. Placed on the video output device child handle that is actively displaying output.
EdidDiscovered	EDID Discovered Protocol from the UEFI 2.0 specification. This protocol is placed on the video output device child handle.
EdidOverride	EDID Override Protocol from the UEFI 2.0 specification. Allow platform to provide EDID information to the producer of the Graphics Output protocol.
GraphicsOutput	Graphics Output Protocol from the UEFI 2.0 specification. Abstraction of a very simple graphics device.
PlatformLogo	The Platform Logo Protocol defines the interface to get the Platform logo image with the display attribute.

6.3 HUMAN INTERFACE INFRASTRUCTURE (HII) & FORMS

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
FormBrowser2	This protocol is defined in UEFI spec. The EFI_FORM_BROWSER2_PROTOCOL is the interface to call for drivers to leverage the EFI configuration driver interface.
FormBrowserEx	Extension Form Browser Protocol provides the services that can be used to register the different hot keys for the standard Browser actions described in UEFI specification.
FormBrowserEx2	Extension Form Browser Protocol provides the services that can be used to register the different hot keys for the standard Browser actions described in UEFI specification.
HiiConfigAccess	The EFI HII results processing protocol invokes this type of protocol when it needs to forward results to a driver's configuration handler. This protocol is published by drivers providing and requesting configuration data from HII.
HiiConfigKeyword	The file provides the mechanism to set and get the values associated with a keyword exposed through a x-UEFI- prefixed configuration language namespace.
HiiConfigRouting	The file provides services to manage the movement of configuration data from drivers to configuration applications. It then serves as the single point to receive configuration information from configuration applications, routing the results to the appropriate drivers.
HiiDatabase	The file provides Database manager for HII-related data structures.
HiiFont	The file provides services to retrieve font information.
HiiImage	The file provides services to access to images in the images database.
HiiImageDecoder	This protocol provides generic image decoder interfaces to various image formats. (C) Copyright 2016 Hewlett Packard Enterprise Development LP .
HiiImageEx	Protocol which allows access to the images in the images database. (C) Copyright 2016-2018 Hewlett Packard Enterprise Development LP SPDX-License-Identifier: BSD-2-Clause-Patent.
HiiPackageList	EFI_HII_PACKAGE_LIST_PROTOCOL as defined in UEFI 2.1. Boot service LoadImage() installs EFI_HII_PACKAGE_LIST_PROTOCOL on the handle if the image contains a custom PE/COFF resource with the type 'HII'.
HiiPopup	This protocol provides services to display a popup window. The protocol is typically produced by the forms browser and consumed by a driver callback handler.
HiiString	The file provides services to manipulate string data.

6.4 BOOT, IMAGE LOADING & DEVICE PATH

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Bds	Boot Device Selection Architectural Protocol as defined in PI spec Volume 2 DXE When the DXE core is done it calls the BDS via this protocol.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Bis	The EFI_BIS_PROTOCOL is used to check a digital signature of a data block against a digital certificate for the purpose of an integrity and authorization check.
BootManagerPolicy	Boot Manager Policy Protocol as defined in UEFI Specification. This protocol is used by EFI Applications to request the UEFI Boot Manager to connect devices using platform policy.
DeferredImageLoad	UEFI 2.2 Deferred Image Load Protocol definition. This protocol returns information about images whose load was denied because of security considerations.
DevicePath	The device path protocol as defined in UEFI 2.0. The device path represents a programmatic path to a device, from a software point of view.
DevicePathFromText	EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL as defined in UEFI 2.0. This protocol provides service to convert text to device paths and device nodes.
DevicePathToText	EFI_DEVICE_PATH_TO_TEXT_PROTOCOL as defined in UEFI 2.0. This protocol provides service to convert device nodes and paths to text.
DevicePathUtilities	EFI_DEVICE_PATH_UTILITIES_PROTOCOL as defined in UEFI 2.0. Use to create and manipulate device paths and device nodes.
FileExplorer	This file explorer protocol defines defines a set of interfaces for how to do file explorer.
LoadFile	Load File protocol as defined in the UEFI 2.0 specification. The load file protocol exists to supports the addition of new boot devices, and to support booting from devices that do not map well to file system.
LoadFile2	Load File protocol as defined in the UEFI 2.0 specification. Load file protocol exists to supports the addition of new boot devices, and to support booting from devices that do not map well to file system.
LoadPe32Image	Load Pe32 Image protocol enables loading and unloading EFI images into memory and executing those images. This protocol uses File Device Path to get an EFI image.
LoadedImage	UEFI 2.0 Loaded image protocol definition. Every EFI driver and application is passed an image handle when it is loaded.
PeCoffImageEmulator	Copyright (c) 2019, Linaro, Ltd. All rights reserved. SPDX-License-Identifier: BSD-2-Clause-Patent.
PlatformBootManager	Copyright (c) 2019, NVIDIA CORPORATION. All rights reserved.
Shell	EFI Shell protocol as defined in the UEFI Shell 2.0 specification including errata. (C) Copyright 2014 Hewlett-Packard Development Company, L.P. .
ShellDynamicCommand	EFI Shell Dynamic Command registration protocol (C) Copyright 2012-2014 Hewlett-Packard Development Company, L.P. .
ShellParameters	EFI Shell protocol as defined in the UEFI Shell 2.0 specification.

6.5 NVRAM VARIABLE SERVICES

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
SmmVarCheck	SMM variable check definitions, it reuses the interface definitions of variable check.
SmmVariable	EFI SMM Variable Protocol is related to EDK II-specific implementation of variables and intended for use as a means to store data in the EFI SMM environment.
VarCheck	Variable check definitions.
Variable	Variable Architectural Protocol as defined in PI Specification VOLUME 2 DXE This provides the services required to get and set environment variables. This protocol must be produced by a runtime DXE driver and may be consumed only by the DXE Foundation.
VariableLock	Variable Lock Protocol is related to EDK II-specific implementation of variables and intended for use as a means to mark a variable read-only after the event EFI_END_OF_DXE_EVENT_GUID is signaled.
VariablePolicy	(no description found).

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
VariableWrite	Variable Write Architectural Protocol as defined in PI Specification VOLUME 2 DXE This provides the services required to set nonvolatile environment variables. This protocol must be produced by a runtime DXE driver and may be consumed only by the DXE Foundation.

6.6 FIRMWARE TABLES (ACPI / SMBIOS)

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
AcpiSystemDescriptionTable	This protocol provides services for creating ACPI system description tables.
AcpiTable	The file provides the protocol to install or remove an ACPI table from a platform.
Smbios	SMBIOS Protocol as defined in PI.2 Specification VOLUME 5 Standard. SMBIOS protocol allows consumers to log SMBIOS data records, and enables the producer to create the SMBIOS tables for a platform.

6.7 BLOCK / DISK / FILE STORAGE

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
AtaAtapiPolicy	ATA ATAPI Policy protocol is produced by platform and consumed by AtaAtapi-PassThruDxe driver.
AtaPassThru	The EFI_ATA_PASS_THRU_PROTOCOL provides information about an ATA controller and the ability to send ATA Command Blocks to any ATA device attached to that ATA controller. The information includes the attributes of the ATA controller.
BlockIo	Block IO protocol as defined in the UEFI 2.0 specification. The Block IO protocol is used to abstract block devices like hard drives, DVD-ROMs and floppy drives.
BlockIo2	Block IO2 protocol as defined in the UEFI 2.3.1 specification. The Block IO2 protocol defines an extension to the Block IO protocol which enables the ability to read and write data at a block level in a non-blocking manner.
BlockIoCrypto	The UEFI Inline Cryptographic Interface protocol provides services to abstract access to inline cryptographic capabilities.
DiskInfo	Provides the basic interfaces to abstract platform information regarding an IDE controller.
DiskIo	Disk IO protocol as defined in the UEFI 2.0 specification. The Disk IO protocol is used to convert block oriented devices into byte oriented devices.
DiskIo2	Disk I/O 2 protocol as defined in the UEFI 2.4 specification. The Disk I/O 2 protocol defines an extension to the Disk I/O protocol to enable non-blocking / asynchronous byte-oriented disk operation.
EraseBlock	This file defines the EFI Erase Block Protocol.
FaultTolerantWrite	Fault Tolerant Write protocol provides boot-time service for fault tolerant write capability for block devices. The protocol provides for non-volatile storage of the intermediate data and private information a caller would need to recover from a critical fault, such as a power failure.
IdeControllerInit	This file declares EFI IDE Controller Init Protocol The EFI_IDE_CONTROLLER_INIT_PROTOCOL provides the chipset-specific information to the driver entity. This protocol is mandatory for IDE controllers if the IDE devices behind the controller are to be enumerated by a driver entity.
MediaSanitize	This file defines the Media Sanitize Protocol.
Nvdimmlabel	EFI NVDIMM Label Protocol Definition The EFI NVDIMM Label Protocol is used to Provides services that allow management of labels contained in a Label Storage Area that are associated with a specific NVDIMM Device Path.
NvmExpressPassthru	This protocol provides services that allow NVM Express commands to be sent to an NVM Express controller or to a specific namespace in a NVM Express controller. This protocol interface is optimized for storage.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
PartitionInfo	This file defines the EFI Partition Information Protocol.
RamDisk	This file defines the EFI RAM Disk Protocol.
ScsiIo	EFI_SCSI_IO_PROTOCOL as defined in UEFI 2.0. This protocol is used by code, typically drivers, running in the EFI boot services environment to access SCSI devices.
ScsiPassThruExt	EFI_EXT_SCSI_PASS_THRU_PROTOCOL as defined in UEFI 2.0. This protocol provides services that allow SCSI Pass Thru commands to be sent to SCSI devices attached to a SCSI channel.
SdMmcOverride	Protocol to describe overrides required to support non-standard SDHCI implementations.
SdMmcPassThru	The EFI_SD_MMC_PASS_THRU_PROTOCOL provides the ability to send SD/MMC Commands to any SD/MMC device attached to the SD compatible pci host controller.
SimpleFileSystem	SimpleFileSystem protocol as defined in the UEFI 2.0 specification. The SimpleFileSystem protocol is the programmatic access to the FAT (12,16,32) file system specified in UEFI 2.0.
SmmFaultTolerantWrite	SMM Fault Tolerant Write protocol is related to EDK II-specific implementation of FTW, provides boot-time service for fault tolerant write capability for block devices in EFI SMM environment. The protocol provides for non-volatile storage of the intermediate data and private information a caller would need to recover from a critical fault, such as a power failure.
SmmSwapAddressRange	The EFI_SMM_SWAP_ADDRESS_RANGE_PROTOCOL is related to EDK II-specific implementation and used to abstract the swap operation of boot block and backup block of FV in EFI SMM environment. This swap is especially needed when updating the boot block of FV.
StorageSecurityCommand	EFI Storage Security Command Protocol as defined in UEFI 2.3.1 specification. This protocol is used to abstract mass storage devices to allow code running in the EFI boot services environment to send security protocol commands to mass storage devices without specific knowledge of the type of device or controller that manages the device.
SwapAddressRange	The EFI_SWAP_ADDRESS_RANGE_PROTOCOL is used to abstract the swap operation of boot block and backup block of FV. This swap is especially needed when updating the boot block of FV.
TapeIo	EFI_TAPE_IO_PROTOCOL as defined in the UEFI 2.0. Provide services to control and access a tape device.
UfsDeviceConfig	This file defines the EFI UFS Device Config Protocol.
UfsHostController	EDKII Universal Flash Storage Host Controller Protocol.
UfsHostControllerPlatform	EDKII_UFS_HC_PLATFORM_PROTOCOL definition.

6.8 PCI, BUS & LOCAL DEVICE ACCESS

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
CpuIo2	This files describes the CPU I/O 2 Protocol. This protocol provides an I/O abstraction for a system processor.
CxlIo	EFI CXL I/O Protocol provides the interfaces to interact with the CXL-specific subsystems of CXL devices. Other interactions with CXL devices should be routed through EFI_PCI_IO_PROTOCOL.
I2cBusConfigurationManagement	I2C Bus Configuration Management Protocol as defined in the PI 1.3 specification. The EFI I2C bus configuration management protocol provides platform specific services that allow the I2C host protocol to reconfigure the switches and multiplexers and set the clock frequency for the I2C bus.
I2cEnumerate	I2C Device Enumerate Protocol as defined in the PI 1.3 specification. This protocol supports the enumerations of device on the I2C bus.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
I2cHost	I2C Host Protocol as defined in the PI 1.3 specification. This protocol provides callers with the ability to do I/O transactions to all of the devices on the I2C bus.
I2cIo	I2C I/O Protocol as defined in the PI 1.3 specification. The EFI I2C I/O protocol enables the user to manipulate a single I2C device independent of the host controller and I2C design.
I2cMaster	I2C Master Protocol as defined in the PI 1.3 specification. This protocol manipulates the I2C host controller to perform transactions as a master on the I2C bus using the current state of any switches or multiplexers in the I2C bus.
IncompatiblePciDeviceSupport	This file declares Incompatible PCI Device Support Protocol Allows the PCI bus driver to support resource allocation for some PCI devices that do not comply with the PCI Specification.
IsaHc	ISA HC Protocol as defined in the PI 1.2.1 specification. This protocol provides registration for ISA devices on a positive- or subtractive-decode ISA bus.
NonDiscoverableDevice	Protocol to describe devices that are not on a discoverable bus.
PciEnumerationComplete	PCI Enumeration Complete Protocol as defined in the PI 1.1 specification. This protocol indicates that pci enumeration complete.
PciHostBridgeResourceAllocation	This file declares PCI Host Bridge Resource Allocation Protocol which provides the basic interfaces to abstract a PCI host bridge resource allocation. This protocol is mandatory if the system includes PCI devices.
PciHotPlugInit	This file declares EFI PCI Hot Plug Init Protocol. This protocol provides the necessary functionality to initialize the Hot Plug Controllers (HPCs) and the buses that they control.
PciHotPlugRequest	Provides services to notify the PCI bus driver that some events have happened in a hot-plug controller (such as a PC Card socket, or PHPC), and to ask the PCI bus driver to create or destroy handles for PCI-like devices. A hot-plug capable PCI bus driver should produce the EFI PCI Hot Plug Request protocol.
PciIo	EFI PCI I/O Protocol provides the basic Memory, I/O, PCI configuration, and DMA interfaces that a driver uses to access its PCI controller.
PciOverride	This file declares EFI PCI Override protocol which provides the interface between the PCI bus driver/PCI Host Bridge Resource Allocation driver and an implementation's driver to describe the unique features of a platform. This protocol is optional.
PciPlatform	This file declares PlatformOpRom protocols that provide the interface between the PCI bus driver/PCI Host Bridge Resource Allocation driver and a platform-specific driver to describe the unique features of a platform. This protocol is optional.
PciRootBridgeIo	PCI Root Bridge I/O protocol as defined in the UEFI 2.0 specification. PCI Root Bridge I/O protocol is used by PCI Bus Driver to perform PCI Memory, PCI I/O, and PCI Configuration cycles on a PCI Root Bridge.
SmbusHc	The file provides basic SMBus host controller management and basic data transactions over the SMBus.
SpiConfiguration	This file defines the SPI Configuration Protocol.
SpiHc	This file defines the SPI Host Controller Protocol.
SpiIo	This file defines the SPI I/O Protocol.
SpiNorFlash	This file defines the SPI NOR Flash Protocol.
SpiSmmConfiguration	This file defines the SPI SMM Configuration Protocol.
SpiSmmHc	This file defines the SPI SMM Host Controller Protocol.
SpiSmmNorFlash	This file defines the SPI SMM NOR Flash Protocol.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
SuperIo	The Super I/O Protocol is installed by the Super I/O driver. The Super I/O driver is a UEFI driver model compliant driver.
SuperIoControl	The Super I/O Control Protocol is installed by the Super I/O driver. It provides the low-level services for SIO devices that enable them to be used in the UEFI driver model.

6.9 USB

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Usb2HostController	EFI_USB2_HC_PROTOCOL as defined in UEFI 2.0. The USB Host Controller Protocol is used by code, typically USB bus drivers, running in the EFI boot services environment, to perform data transactions over a USB bus.
UsbEthernetProtocol	Header file contains code for USB Ethernet Protocol definitions.
UsbFunctionIo	The USB Function Protocol provides an I/O abstraction for a USB Controller operating in Function mode (also commonly referred to as Device, Peripheral, or Target mode) and the mechanisms by which the USB Function can communicate with the USB Host. It is used by other UEFI drivers or applications to perform data transactions and basic USB controller management over a USB Function port.
UsbIo	EFI Usb I/O Protocol as defined in UEFI specification. This protocol is used by code, typically drivers, running in the EFI boot services environment to access USB devices like USB keyboards, mice and mass storage devices.

6.10 BLUETOOTH & WIRELESS

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
BluetoothAttribute	EFI Bluetooth Attribute Protocol as defined in UEFI 2.7. This protocol provides service for Bluetooth ATT (Attribute Protocol) and GATT (Generic Attribute Profile) based protocol interfaces.
BluetoothConfig	EFI Bluetooth Configuration Protocol as defined in UEFI 2.7. This protocol abstracts user interface configuration for Bluetooth device.
BluetoothHc	EFI Bluetooth Host Controller Protocol as defined in UEFI 2.5. This protocol abstracts the Bluetooth host controller layer message transmit and receive.
BluetoothIo	EFI Bluetooth IO Service Binding Protocol as defined in UEFI 2.5. EFI Bluetooth IO Protocol as defined in UEFI 2.5.
BluetoothLeConfig	EFI Bluetooth LE Config Protocol as defined in UEFI 2.7. This protocol abstracts user interface configuration for BluetoothLe device.
Eap	EFI EAP(Extended Authentcaton Protocol) Protocol Definition The EFI EAP Protocol is used to abstract the ability to configure and extend the EAP framework. The definitions in this file are defined in UEFI Specification 2.3.1B, which have not been verified by one implementation yet.
EapConfiguration	This file defines the EFI EAP Configuration protocol.
EapManagement	EFI EAP Management Protocol Definition The EFI EAP Management Protocol is designed to provide ease of management and ease of test for EAPOL state machine. It is intended for the supplicant side.
EapManagement2	This file defines the EFI EAP Management2 protocol.
Supplicant	This file defines the EFI Supplicant Protocol.
WiFi	This file provides management service interfaces of 802.11 MAC layer. It is used by network applications (and drivers) to establish wireless connection with an access point (AP).
WiFi2	This file defines the EFI Wireless MAC Connection II Protocol.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
WiFiProfileSyncProtocol	WiFi profile sync protocol. Supports One Click Recovery or KVM OS recovery boot flow over WiFi.

6.11 NETWORKING – LINK, IP & TRANSPORT

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
AdapterInformation	EFI Adapter Information Protocol definition. The EFI Adapter Information Protocol is used to dynamically and quickly discover or set device information for an adapter.
Arp	EFI ARP Protocol Definition The EFI ARP Service Binding Protocol is used to locate EFI ARP Protocol drivers to create and destroy child of the driver to communicate with other host using ARP protocol. The EFI ARP Protocol provides services to map IP network address to hardware address used by a data link protocol.
Dhcp4	EFI_DHCP4_PROTOCOL as defined in UEFI 2.0. EFI_DHCP4_SERVICE_BINDING_PROTOCOL as defined in UEFI 2.0.
Dhcp6	UEFI Dynamic Host Configuration Protocol 6 Definition, which is used to get IPv6 addresses and other configuration parameters from DHCPv6 servers.
Dns4	This file defines the EFI Domain Name Service Binding Protocol interface. It is split into the following two main sections: DNSv4 Service Binding Protocol (DNSv4SB) DNSv4 Protocol (DNSv4).
Dns6	This file defines the EFI DNSv6 (Domain Name Service version 6) Protocol. It is split into the following two main sections: DNSv6 Service Binding Protocol (DNSv6SB) DNSv6 Protocol (DNSv6).
Dpc	EFI Deferred Procedure Call Protocol.
Ftp4	EFI FTPv4 (File Transfer Protocol version 4) Protocol Definition The EFI FTPv4 Protocol is used to locate communication devices that are supported by an EFI FTPv4 Protocol driver and to create and destroy instances of the EFI FTPv4 Protocol child protocol driver that can use the underlying communication device. The definitions in this file are defined in UEFI Specification 2.3, which have not been verified by one implementation yet.
Ip4	This file defines the EFI IPv4 (Internet Protocol version 4) Protocol interface. It is split into the following three main sections: - EFI IPv4 Service Binding Protocol - EFI IPv4 Variable (deprecated in UEFI 2.4B) - EFI IPv4 Protocol.
Ip4Config2	This file provides a definition of the EFI IPv4 Configuration II Protocol.
Ip6	This file defines the EFI IPv6 (Internet Protocol version 6) Protocol interface. It is split into the following three main sections: - EFI IPv6 Service Binding Protocol - EFI IPv6 Variable (deprecated in UEFI 2.4B) - EFI IPv6 Protocol The EFI IPv6 Protocol provides basic network IPv6 packet I/O services, which includes support for Neighbor Discovery Protocol (ND), Multicast Listener Discovery Protocol (MLD), and a subset of the Internet Control Message Protocol (ICMPv6).
Ip6Config	This file provides a definition of the EFI IPv6 Configuration Protocol.
IpSec	EFI IPSEC Protocol Definition The EFI_IPSEC_PROTOCOL is used to abstract the ability to deal with the individual packets sent and received by the host and provide packet-level security for IP datagram. The EFI_IPSEC2_PROTOCOL is used to abstract the ability to deal with the individual packets sent and received by the host and provide packet-level security for IP datagram.
IpSecConfig	EFI IPsec Configuration Protocol Definition The EFI_IPSEC_CONFIG_PROTOCOL provides the mechanism to set and retrieve security and policy related information for the EFI IPsec protocol driver.
ManagedNetwork	EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL as defined in UEFI 2.0. EFI_MANAGED_NETWORK_PROTOCOL as defined in UEFI 2.0.
Mtftp4	EFI Multicast Trivial File Transfer Protocol Definition.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Mtftp6	UEFI Multicast Trivial File Transfer Protocol v6 Definition, which is built upon the EFI UDPv6 Protocol and provides basic services for client-side unicast and/or multi-cast TFTP operations.
NetworkInterfaceIdentifier	EFI Network Interface Identifier Protocol.
SimpleNetwork	The EFI_SIMPLE_NETWORK_PROTOCOL provides services to initialize a network interface, transmit packets, receive packets, and close a network interface. Basic network device abstraction.
Tcp4	EFI TCPv4(Transmission Control Protocol version 4) Protocol Definition The EFI TCPv4 Service Binding Protocol is used to locate EFI TCPv4 Protocol drivers to create and destroy child of the driver to communicate with other host using TCP protocol. The EFI TCPv4 Protocol provides services to send and receive data stream.
Tcp6	EFI TCPv6(Transmission Control Protocol version 6) Protocol Definition The EFI TCPv6 Service Binding Protocol is used to locate EFI TCPv6 Protocol drivers to create and destroy child of the driver to communicate with other host using TCP protocol. The EFI TCPv6 Protocol provides services to send and receive data stream.
Udp4	UDP4 Service Binding Protocol as defined in UEFI specification. The EFI UDPv4 Protocol provides simple packet-oriented services to transmit and receive UDP packets.
Udp6	The EFI UDPv6 (User Datagram Protocol version 6) Protocol Definition, which is built upon the EFI IPv6 Protocol and provides simple packet-oriented services to transmit and receive UDP packets.
VlanConfig	EFI VLAN Config protocol is to provide manageability interface for VLAN configuration.

6.12 NETWORKING – BOOT, HTTP & HIGHER-LEVEL PROTOCOLS

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Http	This file defines the EFI HTTP Protocol interface. It is split into the following two main sections: HTTP Service Binding Protocol (HTTSPB) HTTP Protocol (HTTP).
HttpBootCallback	This file defines the EFI HTTP Boot Callback Protocol interface.
HttpCallback	This file defines the EDKII HTTP Callback Protocol interface.
HttpUtilities	EFI HTTP Utilities protocol provides a platform independent abstraction for HTTP message comprehension.
IScsiInitiatorName	EFI_ISCSI_INITIATOR_NAME_PROTOCOL as defined in UEFI 2.0. It provides the ability to get and set the iSCSI Initiator Name.
PxeBaseCode	EFI PXE Base Code Protocol definitions, which is used to access PXE-compatible devices for network access and network booting.
PxeBaseCodeCallBack	It is invoked when the PXE Base Code Protocol is about to transmit, has received, or is waiting to receive a packet.
RedfishDiscover	This file defines the EFI Redfish Discover Protocol interface. (C) Copyright 2021 Hewlett Packard Enterprise Development LP SPDX-License-Identifier: BSD-2-Clause-Patent.
Rest	This file defines the EFI REST Protocol interface.
RestEx	This file defines the EFI REST EX Protocol interface. It is split into the following two main sections.
RestJsonStructure	This file defines the EFI REST JSON Structure Protocol interface. (C) Copyright 2020 Hewlett Packard Enterprise Development LP SPDX-License-Identifier: BSD-2-Clause-Patent.
Tls	EFI TLS Protocols as defined in UEFI 2.5. The EFI TLS Service Binding Protocol is used to locate EFI TLS Protocol drivers to create and destroy child of the driver to communicate with other host using TLS protocol.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
TlsConfig	EFI TLS Configuration Protocol as defined in UEFI 2.5. The EFI TLS Configuration Protocol provides a way to set and get TLS configuration.

6.13 SECURITY, MEASURED/SECURE BOOT & CRYPTO

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
AuthenticationInfo	EFI_AUTHENTICATION_INFO_PROTOCOL as defined in UEFI 2.0. This protocol is used on any device handle to obtain authentication information associated with the physical or logical device.
CcMeasurement	If CC Guest firmware supports measurement and an event is created, CC Guest firmware is designed to report the event log with the same data structure in TCG-Platform-Firmware-Profile specification with EFI_TCG2_EVENT_LOG_FORMAT_TCG_2 format. The CC Guest firmware supports measurement, the CC Guest Firmware is designed to produce EFI_CC_MEASUREMENT_PROTOCOL with new GUID EFI_CC_MEASUREMENT_PROTOCOL_GUID to report event log and provides hash capability.
DeviceSecurity	Device Security Protocol definition. It is used to authenticate a device based upon the platform policy.
DeviceSecurityPolicy	Platform Device Security Policy Protocol definition.
Hash	EFI_HASH_SERVICE_BINDING_PROTOCOL as defined in UEFI 2.0. EFI_HASH_PROTOCOL as defined in UEFI 2.0.
Hash2	EFI_HASH2_SERVICE_BINDING_PROTOCOL as defined in UEFI 2.5. EFI_HASH2_PROTOCOL as defined in UEFI 2.5.
Kms	The Key Management Service (KMS) protocol as defined in the UEFI 2.3.1 specification is to provides services to generate, store, retrieve, and manage cryptographic keys. The intention is to specify a simple generic protocol that could be used for many implementations.
Pkcs7Verify	EFI_PKCS7_VERIFY_PROTOCOL as defined in UEFI 2.5. The EFI_PKCS7_VERIFY_PROTOCOL is used to verify data signed using PKCS#7 formatted authentication.
Rng	EFI_RNG_PROTOCOL as defined in UEFI 2.4. The UEFI Random Number Generator Protocol is used to provide random bits for use in applications, or entropy for seeding other random number generators.
Security	Security Architectural Protocol as defined in PI Specification VOLUME 2 DXE Used to provide Security services. Specifically, depending upon the authentication state of a discovered driver in a Firmware Volume, the portable DXE Core Dispatcher will call into the Security Architectural Protocol (SAP) with the authentication state of the driver.
Security2	Security2 Architectural Protocol as defined in PI Specification 1.2.1 VOLUME 2 DXE Abstracts security-specific functions from the DXE Foundation of UEFI Image Verification, Trusted Computing Group (TCG) measured boot, and User Identity policy for image loading and consoles. This protocol must be produced by a boot service or runtime DXE driver.
SecurityPolicy	Security Policy protocol as defined in PI Specification VOLUME 2 DXE.
SmartCardEdge	The Smart Card Edge Protocol provides an abstraction for device to provide Smart Card support. This protocol allows UEFI applications to interface with a Smart Card during boot process for authentication or data signing/decryption, especially if the application has to make use of PKI.
SmartCardReader	The UEFI Smart Card Reader Protocol provides an abstraction for device to provide smart card reader support. This protocol is very close to Part 5 of PC/SC workgroup specifications and provides an API to applications willing to communicate with a smart card or a smart card reader.
Tcg2Protocol	TPM2 Protocol as defined in TCG PC Client Platform EFI Protocol Specification Family “2.0”. See http://trustedcomputinggroup.org for the latest specification.
TcgService	TCG Service Protocol as defined in TCG_EFI_Protocol_1_22_Final See http://trustedcomputinggroup.org for the latest specification.
TrEEProtocol	This protocol is defined to abstract TPM2 hardware access in boot phase.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
UserCredential1	UEFI 2.2 User Credential Protocol definition. It has been removed from UEFI 2.3.1 and replaced by EFI_USER_CREDENTIAL2_PROTOCOL. Attached to a device handle, this protocol identifies a single means of identifying the user.
UserCredential2	UEFI 2.3.1 User Credential Protocol definition. Attached to a device handle, this protocol identifies a single means of identifying the user.
UserManager	UEFI User Manager Protocol definition. This protocol manages user profiles.

6.14 FIRMWARE UPDATE, VOLUMES & CAPSULES

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Capsule	Capsule Architectural Protocol as defined in PI1.0a Specification VOLUME 2 DXE The DXE Driver that produces this protocol must be a runtime driver. The driver is responsible for initializing the CapsuleUpdate() and QueryCapsuleCapabilities() fields of the UEFI Runtime Services Table.
Decompress	The Decompress Protocol Interface as defined in UEFI spec.
EsrtManagement	The Esrt Management Protocol used to register/set/update an updatable firmware resource .
FirmwareManagement	UEFI Firmware Management Protocol definition Firmware Management Protocol provides an abstraction for device to provide firmware management support. The base requirements for managing device firmware images include identifying firmware image revision level and programming the image into the device.
FirmwareManagementProgress	EDK II Firmware Management Progress Protocol.
FirmwareVolume2	The Firmware Volume Protocol provides file-level access to the firmware volume. Each firmware volume driver must produce an instance of the Firmware Volume Protocol if the firmware volume is to be visible to the system during the DXE phase.
FirmwareVolumeBlock	This file provides control over block-oriented firmware devices.
GuidedSectionExtraction	If a GUID-defined section is encountered when doing section extraction, the section extraction driver calls the appropriate instance of the GUIDed Section Extraction Protocol to extract the section stream contained therein.
LegacyRegion2	The Legacy Region Protocol controls the read, write and boot-lock attributes for the region 0xC0000 to 0xFFFFF.
LegacySpiController	This file defines the Legacy SPI Controller Protocol.
LegacySpiFlash	This file defines the Legacy SPI Flash Protocol.
LegacySpiSmmController	This file defines the Legacy SPI SMM Controller Protocol.
LegacySpiSmmFlash	This file defines the Legacy SPI SMM Flash Protocol.
LockBox	LockBox protocol header file. This is used to resolve dependency problem.
Pcd	Native Platform Configuration Database (PCD) Protocol Different with the EFI_PCD_PROTOCOL defined in PI 1.2 specification, the native PCD protocol provide interfaces for dynamic and dynamic-ex type PCD. The interfaces in dynamic type PCD do not require the token space guid as parameter, but interfaces in dynamic-ex type PCD require token space guid as parameter.
PcdInfo	Native Platform Configuration Database (PCD) INFO PROTOCOL. The protocol that provides additional information about items that reside in the PCD database.
PiPcd	Platform Configuration Database (PCD) Protocol defined in PI 1.2 Vol3 A platform database that contains a variety of current platform settings or directives that can be accessed by a driver or application. PI PCD protocol only provide the accessing interfaces for Dynamic-Ex type PCD.
PiPcdInfo	Platform Configuration Database (PCD) Info Protocol defined in PI 1.2.1 Vol3. The protocol that provides additional information about items that reside in the PCD database.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
SmmFirmwareVolumeBlock	SMM Firmware Volume Block protocol is related to EDK II-specific implementation of FVB driver, provides control over block-oriented firmware devices and is intended to use in the EFI SMM environment.

6.15 TIMERS, CLOCK, RESET & POWER

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Metronome	Metronome Architectural Protocol as defined in PI SPEC VOLUME 2 DXE This code abstracts the DXE core to provide delay services.
MonotonicCounter	Monotonic Counter Architectural Protocol as defined in PI SPEC VOLUME 2 DXE This code provides the services required to access the system's monotonic counter.
PlatformSpecificResetFilter	This Protocol provides services to register a platform specific reset filter for ResetSystem(). A reset filter evaluates the parameters passed to ResetSystem() and converts a ResetType of EfiResetPlatformSpecific to a non-platform specific reset type.
PlatformSpecificResetHandler	This protocol provides services to register a platform specific handler for ResetSystem(). The registered handlers are called after the UEFI 2.7 Reset Notifications are processed.
RealTimeClock	Real Time clock Architectural Protocol as defined in PI Specification VOLUME 2 DXE This code abstracts time and data functions. Used to provide Time and date related EFI runtime services.
Reset	Reset Architectural Protocol as defined in PI Specification VOLUME 2 DXE Used to provide ResetSystem runtime services The ResetSystem () UEFI 2.0 service is added to the EFI system table and the EFI_RESET_ARCH_PROTOCOL_GUID protocol is registered with a NULL pointer.
ResetNotification	EFI Reset Notification Protocol as defined in UEFI 2.7. This protocol provides services to register for a notification when ResetSystem is called.
Timer	Timer Architectural Protocol as defined in PI Specification VOLUME 2 DXE This code is used to provide the timer tick for the DXE core.
Timestamp	EFI Timestamp Protocol as defined in UEFI2.4 Specification. Used to provide a platform independent interface for retrieving a high resolution timestamp counter.
WatchdogTimer	Watchdog Timer Architectural Protocol as defined in PI Specification VOLUME 2 DXE Used to provide system watchdog timer services.

6.16 CPU & MULTI-PROCESSOR

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
Cpu	CPU Architectural Protocol as defined in PI spec Volume 2 DXE This code abstracts the DXE core from processor implementation details.
DebuggerConfiguration	EBC Debugger configuration protocol.
DtFixup	Device Tree Fixup Protocol. Modifies a device tree in memory to align it with the firmware's view of the platform.
Ebc	Describes the protocol interface to the EBC interpreter.
EbcSimpleDebugger	EBC Simple Debugger protocol for debug EBC code.
EbcVmTest	EBC VM Test protocol for test purposes.
GenericMemoryTest	This protocol defines the generic memory test interfaces in Dxe phase.
IoMmu	EFI IOMMU Protocol.
MemoryAccept	The file provides the protocol to provide interface to accept memory.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
MemoryAttribute	EFI Memory Attribute Protocol provides retrieval and update service for memory attributes in EFI environment.
MpService	When installed, the MP Services Protocol produces a collection of services that are needed for MP management. The MP Services Protocol provides a generalized way of performing following tasks: - Retrieving information of multi-processor environment and MP-related status of specific processors.

6.17 PI DRIVER MODEL & PLATFORM OVERRIDE (FIRMWARE-INTERNAL)

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
BusSpecificDriverOverride	Bus Specific Driver Override protocol as defined in the UEFI 2.0 specification. Bus drivers that have a bus specific algorithm for matching drivers to controllers are required to produce this protocol for each controller.
ComponentName	EFI Component Name Protocol as defined in the EFI 1.1 specification. This protocol is used to retrieve user readable names of EFI Drivers and controllers managed by EFI Drivers.
ComponentName2	UEFI Component Name 2 Protocol as defined in the UEFI 2.1 specification. This protocol is used to retrieve user readable names of drivers and controllers managed by UEFI Drivers.
DriverBinding	UEFI DriverBinding Protocol is defined in UEFI specification. This protocol is produced by every driver that follows the UEFI Driver Model, and it is the central component that allows drivers and controllers to be managed.
DriverConfiguration	EFI Driver Configuration Protocol.
DriverConfiguration2	UEFI Driver Configuration2 Protocol.
DriverDiagnostics	EFI Driver Diagnostics Protocol.
DriverDiagnostics2	UEFI Driver Diagnostics2 Protocol.
DriverFamilyOverride	UEFI Driver Family Protocol.
DriverHealth	EFI Driver Health Protocol definitions. When installed, the Driver Health Protocol produces a collection of services that allow the health status for a controller to be retrieved.
DriverSupportedEfiVersion	The protocol provides information about the version of the EFI specification that a driver is following. This protocol is required for EFI drivers that are on PCI and other plug-in cards.
IpmiProtocol	Protocol of Ipmi for both SMS and SMM.
PlatformDriverOverride	Platform Driver Override protocol as defined in the UEFI 2.1 specification.
PlatformToDriverConfiguration	UEFI Platform to Driver Configuration Protocol is defined in UEFI specification. This is a protocol that is optionally produced by the platform and optionally consumed by a UEFI Driver in its Start() function.
Ps2Policy	PS/2 policy protocol abstracts the specific platform initialization and settings.
ServiceBinding	UEFI Service Binding Protocol is defined in UEFI specification. The file defines the generic Service Binding Protocol functions.
SmmExitBootServices	EDKII SMM Exit Boot Services protocol. This SMM protocol is to be published by the SMM Foundation code to associate with EFI_EVENT_GROUP_EXIT_BOOT_SERVICES to notify SMM driver that system enter exit boot services.
SmmLegacyBoot	EDKII SMM Legacy Boot protocol. This SMM protocol is to be published by the SMM Foundation code to associate with EFI_EVENT_LEGACY_BOOT_GUID to notify SMM driver that system enter legacy boot.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
SmmReadyToBoot	EDKII SMM Ready To Boot protocol. This SMM protocol is to be published by the SMM Foundation code to associate with EFI_EVENT_GROUP_READY_TO_BOOT to notify SMM driver that system enter ready to boot.

6.18 SMM/MM FOUNDATION & DISPATCH (FIRMWARE-INTERNAL, PI SPEC)

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
DxeMmReadyToLock	DXE MM Ready To Lock protocol introduced in the PI 1.5 specification.
DxeSmmReadyToLock	DXE SMM Ready To Lock protocol introduced in the PI 1.2 specification. According to PI 1.4a specification, this UEFI protocol indicates that resources and services that should not be used by the third party code are about to be locked.
MmAccess	EFI MM Access Protocol as defined in the PI 1.5 specification. This protocol is used to control the visibility of the MMRAM on the platform.
MmBase	EFI MM Base Protocol as defined in the PI 1.5 specification. This protocol is utilized by all MM drivers to locate the MM infrastructure services and determine whether the driver is being invoked inside MMRAM or outside of MMRAM.
MmCommunication	EFI MM Communication Protocol as defined in the PI 1.5 specification. This protocol provides a means of communicating between drivers outside of MM and MMI handlers inside of MM.
MmCommunication2	EFI MM Communication Protocol 2 as defined in the PI 1.7 errata A specification. This protocol provides a means of communicating between drivers outside of MM and MMI handlers inside of MM.
MmCommunication3	EFI MM Communication Protocol 3 as defined in the PI 1.9 specification. This protocol provides a means of communicating between drivers outside of MM and MMI handlers inside of MM.
MmConfiguration	EFI MM Configuration Protocol as defined in the PI 1.5 specification. This protocol is used to: 1) report the portions of MMRAM regions which cannot be used for the MMRAM heap.
MmControl	EFI MM Control Protocol as defined in the PI 1.5 specification. This protocol is used initiate synchronous MMI activations.
MmCpu	EFI MM CPU Protocol as defined in the PI 1.5 specification. This protocol allows MM drivers to access architecture-standard registers from any of the CPU save state areas.
MmCpuIo	MM CPU I/O 2 protocol as defined in the PI 1.5 specification. This protocol provides CPU I/O and memory access within MM.
MmEndOfDxe	MM End Of Dxe protocol introduced in the PI 1.5 specification. This protocol is a mandatory protocol published by MM Foundation code.
MmGpiDispatch	MM General Purpose Input (GPI) Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides the parent dispatch service for the General Purpose Input (GPI) MMI source generator.
MmIoTrapDispatch	MM IO Trap Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides a parent dispatch service for IO trap MMI sources.
MmMp	EFI MM MP Protocol is defined in the PI 1.5 specification. The MM MP protocol provides a set of functions to allow execution of procedures on processors that have entered MM.
MmPciRootBridgeIo	MM PCI Root Bridge IO protocol as defined in the PI 1.5 specification. This protocol provides PCI I/O and memory access within MM.
MmPeriodicTimerDispatch	MM Periodic Timer Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides the parent dispatch service for the periodical timer MMI source generator.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
MmPowerButtonDispatch	MM Power Button Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides the parent dispatch service for the power button MMI source generator.
MmReadyToLock	MM Ready To Lock protocol introduced in the PI 1.5 specification. This protocol is a mandatory protocol published by the MM Foundation code when the system is preparing to lock certain resources and interfaces in anticipation of the invocation of 3rd party extensible modules.
MmReportStatusCodeHandler	This protocol provides registering and unregistering services to status code consumers while in DXE MM.
MmStandbyButtonDispatch	MM Standby Button Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides the parent dispatch service for the standby button MMI source generator.
MmStatusCode	EFI MM Status Code Protocol as defined in the PI 1.5 specification. This protocol provides the basic status code services while in MM.
MmSwDispatch	MM Software Dispatch Protocol introduced from PI 1.5 Specification Volume 4 Management Mode Core Interface. This protocol provides the parent dispatch service for a given MMI source generator.
MmSxDispatch	MM Sx Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. Provides the parent dispatch service for a given Sx-state source generator.
MmUsbDispatch	MM USB Dispatch Protocol as defined in PI 1.5 Specification Volume 4 Management Mode Core Interface. Provides the parent dispatch service for the USB MMI source generator.
SmmAccess2	EFI SMM Access2 Protocol as defined in the PI 1.2 specification. This protocol is used to control the visibility of the SMRAM on the platform.
SmmBase2	EFI SMM Base2 Protocol as defined in the PI 1.2 specification. This protocol is utilized by all SMM drivers to locate the SMM infrastructure services and determine whether the driver is being invoked inside SMRAM or outside of SMRAM.
SmmCommunication	EFI SMM Communication Protocol as defined in the PI 1.2 specification. This protocol provides a means of communicating between drivers outside of SMM and SMI handlers inside of SMM.
SmmConfiguration	EFI SMM Configuration Protocol as defined in the PI 1.2 specification. This protocol is used to: 1) report the portions of SMRAM regions which cannot be used for the SMRAM heap.
SmmControl2	EFI SMM Control2 Protocol as defined in the PI 1.2 specification. This protocol is used initiate synchronous SMI activations.
SmmCpu	EFI SMM CPU Protocol as defined in the PI 1.2 specification. This protocol allows SMM drivers to access architecture-standard registers from any of the CPU save state areas.
SmmCpuIo2	SMM CPU I/O 2 protocol as defined in the PI 1.2 specification. This protocol provides CPU I/O and memory access within SMM.
SmmEndOfDxe	SMM End Of Dxe protocol introduced in the PI 1.2.1 specification. According to PI 1.4a specification, this protocol indicates end of the execution phase when all of the components are under the authority of the platform manufacturer.
SmmGpiDispatch2	SMM General Purpose Input (GPI) Dispatch2 Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. This protocol provides the parent dispatch service for the General Purpose Input (GPI) SMI source generator.
SmmIoTrapDispatch2	SMM IO Trap Dispatch2 Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. This protocol provides a parent dispatch service for IO trap SMI sources.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
SmmMemoryAttribute	SMM Memory Attribute Protocol provides retrieval and update service for memory attributes in EFI SMM environment.
SmmPciRootBridgeIo	SMM PCI Root Bridge IO protocol as defined in the PI 1.2 specification. This protocol provides PCI I/O and memory access within SMM.
SmmPeriodicTimerDispatch2	SMM Periodic Timer Dispatch Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. This protocol provides the parent dispatch service for the periodical timer SMI source generator.
SmmPowerButtonDispatch2	SMM Power Button Dispatch2 Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. This protocol provides the parent dispatch service for the power button SMI source generator.
SmmReadyToLock	SMM Ready To Lock protocol introduced in the PI 1.2 specification. According to PI 1.4a specification, this SMM protocol indicates that SMM resources and services that should not be used by the third party code are about to be locked.
SmmReportStatusCodeHandler	This protocol provides registering and unregistering services to status code consumers while in DXE SMM.
SmmStandbyButtonDispatch2	SMM Standby Button Dispatch2 Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. This protocol provides the parent dispatch service for the standby button SMI source generator.
SmmStatusCode	EFI SMM Status Code Protocol as defined in the PI 1.2 specification. This protocol provides the basic status code services while in SMM.
SmmSwDispatch2	SMM Software Dispatch Protocol introduced from PI 1.2 Specification Volume 4 System Management Mode Core Interface. This protocol provides the parent dispatch service for a given SMI source generator.
SmmSxDispatch2	SMM Sx Dispatch Protocol as defined in PI 1.2 Specification Volume 4 System Management Mode Core Interface. Provides the parent dispatch service for a given Sx-state source generator.
SmmUsbDispatch2	SMM USB Dispatch2 Protocol as defined in PI 1.1 Specification Volume 4 System Management Mode Core Interface. Provides the parent dispatch service for the USB SMI source generator.

6.19 REPORT STATUS CODE, DEBUG & MISC UTILITY

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
DebugPort	The file defines the EFI Debugport protocol. This protocol is used by debug agent to communicate with the remote debug host.
DebugSupport	DebugSupport protocol and supporting definitions as defined in the UEFI2.4 specification. The DebugSupport protocol is used by source level debuggers to abstract the processor and handle context save and restore operations.
Print2	Produces EFI_PRINT2_PROTOCOL and EFI_PRINT2S_PROTOCOL. These protocols define basic print functions to print the format unicode and ascii string.
RegularExpressionProtocol	This section defines the Regular Expression Protocol. This protocol is used to match Unicode strings against Regular Expression patterns.
ReportStatusCodeHandler	This protocol provide registering and unregistering services to status code consumers while in DXE.
Runtime	Runtime Architectural Protocol as defined in PI Specification VOLUME 2 DXE Allows the runtime functionality of the DXE Foundation to be contained in a separate driver. It also provides hooks for the DXE Foundation to export information that is needed at runtime.
S3SaveState	S3 Save State Protocol as defined in PI 1.6(Erta A) Specification VOLUME 5 Standard. This protocol is used by DXE PI module to store or record various IO operations to be replayed during an S3 resume.

PROTOCOL	PURPOSE (FROM ITS OWN EDK2 HEADER)
S3SmmSaveState	S3 SMM Save State Protocol as defined in PI1.2 Specification VOLUME 5 Standard. The <code>EFI_S3_SMM_SAVE_STATE_PROTOCOL</code> publishes the PI SMMboot script abstractions. On an S3 resume boot path the data stored via this protocol is replayed in the order it was stored.
StatusCode	Status code Runtime Protocol as defined in PI Specification 1.4a VOLUME 2 DXE.
UnicodeCollation	Unicode Collation protocol that follows the UEFI 2.0 specification. This protocol is used to allow code running in the boot services environment to perform lexical comparison functions on Unicode strings for given languages.

07 PART II — THE INTEL HDA DRIVER

08 WHY A PLATFORM TRAIT

The driver is split into two crates specifically so that none of the register-level, verb-encoding, or codec-discovery logic has to know it's running under UEFI:

CRATE	RESPONSIBILITY
<code>intel-hda</code>	<code>#![no_std]</code> , <code>no_alloc</code> , zero dependencies. Defines the <code>Platform</code> trait (<code>MMIO read/write</code> , <code>delay_us</code> , <code>alloc_dma</code>) and everything built purely against it: register layout (<code>regs</code>), verb encoding (<code>verbs</code>), PCM format math (<code>format</code>), decoded codec parameter shapes (<code>codec</code>), and the <code>Controller<P: Platform></code> that does reset, CORB/RIRB command dispatch, codec/widget discovery, and stream-descriptor programming.
<code>intel-hda-uefi</code>	The one <code>impl Platform</code> : PCI bus-0 scan for class <code>0x04/subclass 0x03</code> , enabling Memory Space + Bus Master, raw volatile MMIO through a pointer derived from <code>BAR0</code> , <code>boot::stall</code> for delays, <code>boot::allocate_pages</code> for DMA memory. Exposes a single <code>open() -> Result<Controller<UefiPlatform>, OpenError></code> .
<code>ruefi</code> (Audio tab)	<code>src/screens/hda.rs</code> — calls <code>intel_hda_uefi::open()</code> , walks the codec tree once at construction to find a playable DAC/pin path, and on Enter builds a sine-wave sample buffer plus a buffer descriptor list, programs the codec and stream descriptor, and starts the stream.

WHAT THIS BUYS

A second backend — a from-scratch kernel, or a hosted test harness that fakes MMIO in a `Vec<u8>` — needs only a new, small `impl Platform`. Every line in `Controller<P>` (reset sequencing, verb encoding, widget walking, stream programming) is untouched and already exercised against real emulated hardware.

09 HARDWARE MODEL, BRIEFLY

An HDA controller exposes a 16KB memory-mapped register block (`BAR0`): global control/status, a pair of DMA-driven command rings (`CORB` outbound, `RIRB` inbound) used to send 32-bit **verbs** to codecs and receive their responses, and one register block per DMA stream, each pointing at a **buffer descriptor list** (BDL) of physical sample-buffer fragments. A codec, addressed over the same link, exposes a tree of nodes — a root, one or more Function Groups, and under an Audio Function Group, widgets: DACs, ADCs, mixers, selectors, and pin complexes (the externally-facing jacks). Routing audio out means: pick a pin, follow its connection list back to a DAC, and program matching stream/format state on both the codec side (via verbs) and the controller side (via the stream descriptor registers).

FULL REGISTER MAP LIVES ELSEWHERE

Every register offset, bit field, the CORB/RIRB entry formats, the verb encoding split (12-bit-verb/8-bit-payload vs. 4-bit-verb/16-bit-payload), and the codec parameter/control tables this driver was built against are in docs/hda-register-map.typ — distilled from the official Intel spec (docs/reference/intel-hda-spec-1.0a.pdf) and cross-checked against the Linux kernel’s hda_verbs.h (docs/reference/linux-hda_verbs.h). This section only summarizes the shape of it.

10

BRING-UP SEQUENCE

STEPWHAT HAPPENS

- 1

intel_hda_uefi::open() scans PCI bus 0 for class 0x04/subclass 0x03, enables Memory Space + Bus Master, and resolves BAR0 to an MMIO base.
- 2

Controller::new resets the controller (GCTL.CRST 0→1, polled), allocates CORB/RIRB DMA buffers, and programs both rings’ base addresses, sizes, and run bits.
- 3

HdaScreen::new reads STATESTS for codec presence, then for each present address: Vendor ID → find the Audio Function Group → walk its widgets for an output-capable Pin Complex → read that pin’s connection list for a DAC.
- 4

On Enter: build a 2-second 48kHz/16-bit/stereo sine buffer and a 2-entry BDL over it, program the DAC’s converter format/stream-channel/amp and the pin’s widget-control/amp/EAPD, program the stream descriptor (format, BDL pointer, cyclic buffer length, last valid index, stream tag), and set RUN .

11

VERIFIED BEHAVIOR

The pipeline above was verified against QEMU’s ich9-intel-hda + hda-duplex by redirecting the emulated codec’s output to a WAV file (-audiodev wav,id=snd0,path=...) and inspecting the samples directly rather than trusting that “no panic” meant “produces audio.”

MEASUREMENT	RESULT
Peak amplitude	7999 / 8000 target — full-scale, undistorted
Nonzero sample coverage	>99.99% of captured frames, sustained across the whole 2-second buffer, looping correctly through the BDL’s two halves
Frequency (zero-crossing count over a 1s window)	439 crossings against a 440 Hz target
QEMU trace confirmation	hda_audio_format st dac, 2 x PCM-S16 @ 48000 Hz and hda_audio_running st dac, nr 1, run 1 — the codec’s own internal state matches exactly what the driver programmed

12

TWO BUGS FOUND AGAINST REAL (EMULATED) HARDWARE

Both were found by instrumenting the driver with temporary register dumps once the first attempt produced “everything succeeds, nothing plays,” then reading QEMU’s own hw/audio/intel-hda.c and hda-codec.c to find the actual gating logic — guessing from the spec text alone would not have caught either one.

RINTCNT GATES COMMAND PROCESSING ON THIS EMULATION

The spec's own text reads as though `RINTCNT` (RIRB response-interrupt count) only controls when the **interrupt flag** gets set. QEMU's model instead exits its CORB-fetch loop outright whenever `responses_since_service == RINTCNT` — and since both start at the same reset value (0), that condition is true before the first command is ever sent, silently blocking the whole command channel forever. **Fix:** program `RINTCNT` to a nonzero value, enable `RIRBCTL`'s interrupt-enable bit even though nothing handles the interrupt, and write a 1 to `RIRBSTS.RINTFL` after consuming every response — a write-1-to-clear that is always safe on real hardware, and the only thing that resets this emulation's internal counter.

AMP_GAIN 0 IS NOT 0 DB WHEN AMPLIFIER CAPABILITIES ISN'T IMPLEMENTED

The Amplifier Capabilities parameter's `Offset` field is what maps to 0 dB — not a literal 0. QEMU's built-in codec doesn't register that parameter at all, so `Get Parameter` on it returns all-zero (`NumSteps = 0`, `Offset = 0`), and driving a widget's gain to that "offset" value sets an explicit, near-silent gain of 0 on any codec that **does** implement variable gain — while being a pointless no-op on one that doesn't. Since a codec's post-reset default is already required to be a sane, unmuted state, the fix is to only issue an explicit Amplifier Gain/Mute **Set** when `NumSteps > 0`, and otherwise leave the widget alone.

13 REFERENCES

- `docs/hda-register-map.typ` — the full Intel HDA register/verb/parameter reference this driver was implemented against.
- `docs/reference/intel-hda-spec-1.0a.pdf` — Intel High Definition Audio Specification, Revision 1.0a (June 17, 2010).
- `docs/reference/linux-hda_verbs.h` — Linux kernel's `AC_VERB_*` / `AC_PAR_*` constants, used to cross-check verb encoding.
- `rust-osdev/uefi-rs` — the `uefi` crate (v0.40) all of Part I is written against.
- QEMU 8.2.2 source, `hw/audio/intel-hda.c` and `hw/audio/hda-codec.c` — read directly to diagnose both bugs in Section 12.